IAC Provising Real Estate System

CourseName:DevOps

InstitutionName: Medicaps University–DatagamiSkillBasedCourse

StudentName(s)&EnrolmentNumber(s):

Srno	StudentName	EnrolmentNumber
1.	KUSHAGRA GUPTA	EN22CS301546
2.	KHUSHI PALIWAL	EN22CS301505
3.	KRITIKA NAGAR	EN22CS301526
4.	KARTIK GUPTA	EN22CS301488
5.	KISHNA SHARMA	EN22CS301510
6.	JAYESH JAIN	EN22CS301465

GroupName:Group07D5

Project Number: DO-07

IndustryMentorName:Mr.Vaibhav

UniversityMentorName:Prof.Ritesh Joshi

Academic Year: 2025-2026

Problem Statement & Objectives

1. Problem Statement

In today's real estate systems, applications are often deployed manually, which makes the process time-consuming, error-prone and difficult to scale.

There is a lack of automation in setting up infrastructure and deploying applications, which leads to inconsistency across environments.

Also, managing different components like frontend, backend and database becomes complex without proper containerization.

Therefore, there is a need to build a system that not only provides property listing functionality but also ensures automated, scalable and reliable deployment using modern DevOps practices

2. Project Objectives

The main objective of this project is to develop a **full stack real estate web application** and deploy it using modern DevOps practices.

Firstly, the goal is to create a user-friendly platform where users can view property listings along with details such as price, location and images.

Secondly, the project aims to design a proper backend system that handles API requests and manages communication with the database efficiently.

Another important objective is to ensure proper data storage and retrieval using a relational database like MySQL.

From a DevOps perspective, the key objective is to **containerize the entire application using Docker**, so that all components run in isolated and consistent environments without dependency issues.

In addition to this, the project also focuses on **automating infrastructure setup using Terraform**, so that cloud resources like EC2 instances and security groups can be created programmatically instead of manually.

Another objective is to deploy the application on AWS cloud and make it accessible via public IP, ensuring real-time availability.

3. Scope of the Project

The main objective of this project is to develop a **full stack real estate web application** and deploy it using modern DevOps practices.

Firstly, the goal is to create a user-friendly platform where users can view property listings along with details such as price, location and images.

Secondly, the project aims to design a proper backend system that handles API requests and manages communication with the database efficiently.

Another important objective is to ensure proper data storage and retrieval using a relational database like MySQL.

From a DevOps perspective, the key objective is to **containerize the entire application using Docker**, so that all components run in isolated and consistent environments without dependency issues.

In addition to this, the project also focuses on **automating infrastructure setup using Terraform**, so that cloud resources like EC2 instances and security groups can be created programmatically instead of manually.

Another objective is to deploy the application on AWS cloud and make it accessible via public IP, ensuring real-time availability.

Finally, the project aims to demonstrate scalability and portability by designing a system that can be easily extended in future with tools like CI/CD pipelines and Kubernetes.

Proposed Solution

1. Keyfeatures

- User-friendly property listing interface
- Full stack web application architecture
- RESTful API-based backend system
- Efficient database management using MySQL
- Containerization using Docker
- Infrastructure as Code (IaC) using Terraform
- Automated cloud resource provisioning (EC2, Security Groups)
- Deployment on AWS with public accessibility
- Secure and configurable environment setup
- Scalable and portable system design
- DevOps-ready architecture with CI/CD support

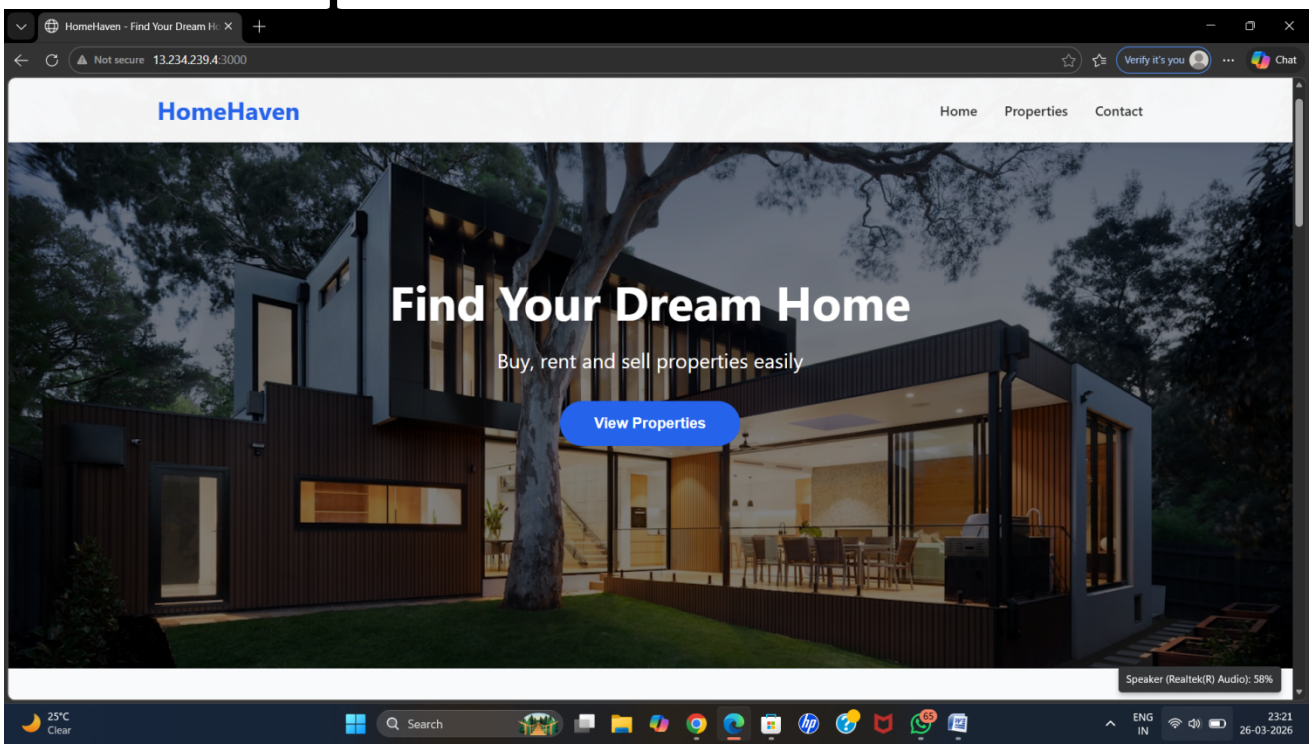
2.Overall Architecture / Workflow

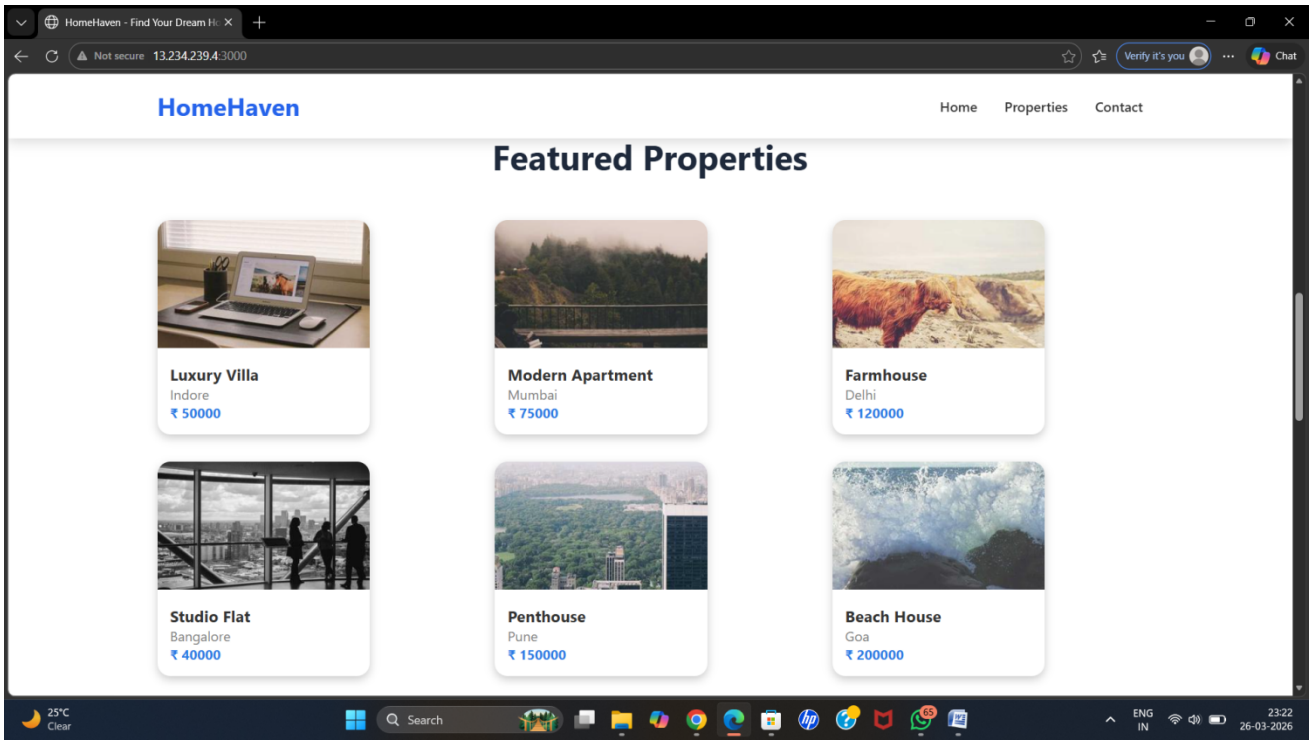
The overall architecture of the IaC-based real estate system follows a DevOps approach where infrastructure provisioning, application deployment, and management are fully automated. The system consists of multiple layers, including the user layer, application layer, orchestration layer, CI/CD pipeline, and infrastructure layer. Users interact with the real estate application through a web interface, while the application itself runs inside Docker containers to ensure portability and consistency. Kubernetes is used as the orchestration tool to manage these containers, handle scaling, and maintain high availability of the system.
The workflow begins with developers writing application and infrastructure code and storing it in a Git repository. Whenever changes are pushed, a CI/CD pipeline (using Jenkins or GitHub Actions) is triggered. The pipeline builds the application, creates Docker images, and performs automated testing. These images are then stored in a container registry. Meanwhile, infrastructure is provisioned automatically using IaC.

4.Tools & Technologies Used

- **Infrastructure as Code (Terraform / Ansible):** Used to automate infrastructure setup and configuration, ensuring consistency and reducing manual effort.
- **Docker:** Packages the application and its dependencies into containers for easy and consistent deployment across environments. .
- **CI/CD Tools (Jenkins / GitHub Actions):** Automate the process of building, testing, and deploying the application whenever code changes occur.
- **Git (GitHub/GitLab):** Used for version control to track code changes and enable team collaboration.
- **Cloud Platform (AWS / Local):** Provides the environment to host and run the application infrastructure.
- **Container Registry (Docker Hub / AWS ECR):** Stores Docker images so they can be easily accessed and deployed by Kubernetes.
- **Shell Scripting / Python:** Helps automate repetitive tasks like setup, deployment, and pipeline operations.

Results & Output





Key Outcome

1. **End-to-End Application Development**
   Successfully developed a complete full stack real estate web application by integrating frontend, backend, and database components. This demonstrates the ability to build a real-world application from scratch.
2. **Successful Cloud Deployment**
   The application was deployed on Amazon Web Services (AWS), making it accessible via a public IP. This ensures real-time availability and demonstrates practical knowledge of cloud hosting.
3. **Automated Infrastructure Provisioning**
   Infrastructure setup was automated using Terraform. This reduced manual effort, minimized configuration errors, and ensured consistent and repeatable deployments.
4. **Improved Deployment Consistency with Containerization**
   Implemented containerization using Docker, which ensured that the application runs consistently across different environments and eliminated dependency-related issues.
5. **Efficient Backend and API Management**
   Developed a structured backend system with RESTful APIs to handle client requests efficiently, ensuring smooth communication between the frontend and database.
6. **Reliable Data Management**
   Used a relational database system (MySQL) to store and manage application data, ensuring data integrity, structured storage, and efficient retrieval.
7. **Enhanced Security Implementation**
   Configured security groups and controlled access to cloud resources, improving the overall security of the deployed application.
8. **Scalability and Future Readiness**
   Designed the system with scalability in mind, allowing future integration with CI/CD pipelines and orchestration tools such as Kubernetes.
9. **Practical DevOps Experience**
   Gained hands-on experience with modern DevOps practices, including containerization, infrastructure as code, and cloud deployment workflows.
10. **Industry-Relevant Skill Development**
    Strengthened skills in full stack development, cloud computing, and DevOps tools, making the project aligned with industry requirements.

Conclusion

The project successfully demonstrates the integration of full stack development with modern DevOps practices to build and deploy a real-world real estate web application. It highlights the ability to design a complete system starting from user interface development to backend logic and database management, ensuring smooth data flow and efficient performance.

A major achievement of the project is the adoption of Infrastructure as Code using Terraform, which automates the provisioning of cloud resources and eliminates the need for manual configuration. This not only improves efficiency but also ensures consistency and reproducibility across deployments. Additionally, the use of Docker for containerization ensures that the application runs reliably in different environments without dependency-related issues.

The deployment on Amazon Web Services (AWS) further strengthens the project by making it accessible, scalable, and aligned with industry standards. The use of cloud infrastructure demonstrates an understanding of real-time application hosting, security configurations, and resource management.

Overall, the project reflects a strong foundation in both development and DevOps domains. It not only delivers a functional application but also showcases the ability to implement scalable, portable, and maintainable solutions. This project serves as a practical example of how modern technologies can be combined to build efficient, automated, and industry-ready systems, making it highly relevant for real-world applications and future enhancements.

FutureScope&Enhancements

1. **Implementation of CI/CD Pipelines**
   The project can be enhanced by integrating continuous integration and continuous deployment pipelines using tools like Jenkins or GitHub Actions. This will automate the build, testing, and deployment process, reducing manual effort and improving reliability.
2. **Container Orchestration with Kubernetes**
   To handle large-scale deployments, the application can be deployed using Kubernetes. This will enable auto-scaling, load balancing, and efficient container management.
3. **Advanced Security Enhancements**
   Security can be improved by implementing authentication and authorization mechanisms such as JWT, role-based access control, and HTTPS encryption. Integration with services like AWS IAM can further enhance access control.
4. **Performance Optimization and Load Balancing**
   The application can be optimized by using caching mechanisms (e.g., Redis) and load balancing techniques. Using AWS Elastic Load Balancing can help distribute traffic efficiently and improve performance.
5. **Microservices Architecture Transition**
   The monolithic architecture can be converted into microservices to improve scalability and maintainability. Each service (user, property, payment) can be independently deployed and managed.
6. **Integration of Monitoring and Logging Tools**
   Monitoring tools like Prometheus and visualization tools like Grafana can be added for real-time performance tracking and system health monitoring.
7. **Enhanced User Features**
   Additional functionalities such as user authentication, property booking, advanced search filters, and recommendation systems can improve user experience.
8. **Mobile Application Development**
   A mobile version of the application can be developed using frameworks like React Native or Flutter to increase accessibility and reach.